

27. Null Object Design Pattern in Java

27.1 Introduction

The **Null Object Pattern** is a **behavioral design pattern** that uses a special object representing a "do nothing" or default behavior instead of using null references.

27.2 Problem it Solves

Avoids NullPointerException and simplifies client code by replacing null with an object that exhibits neutral behavior.

27.3 What is Null Object Pattern?

- Instead of returning null, return an object that conforms to expected interface
- This object does nothing but adheres to interface/contract

27.4 Real-World Analogy

Imagine asking for a **hotel room**. If none are available, instead of giving null, the system provides a **NoRoomAvailable** object that says: "Sorry, no room" but still allows method calls.

27.5 Structure

```
package com.mannemlokes;

public interface Service {
    void execute();
}

class RealService implements Service {
    public void execute() {
        System.out.println("Executing real service.");
    }
}

class NullService implements Service {
    public void execute() {
        // do nothing
    }
}
```

27.6 Benefits

- Avoid null checks in client code
- Follows Liskov Substitution Principle
- Simplifies control flow

27.7 Implementation Steps

1. Define a common interface
2. Create a real implementation
3. Create a null implementation
4. Use factory/provider to return either

27.8 Examples

Example1: Logger Example

```
package com.mannemlokesesh.example1;

interface Logger {
    void log(String message);
}

class ConsoleLogger implements Logger {
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

class NullLogger implements Logger {
    public void log(String message) {
        // do nothing
    }
}

public class Application {
    private Logger logger;

    public Application(Logger logger) {
        this.logger = logger;
    }

    public void run() {
        logger.log("App started.");
        System.out.println("Running application...");
        logger.log("App ended.");
    }

    public static void main(String[] args) {
        Application app1 = new Application(new ConsoleLogger());
        app1.run();

        System.out.println("\n--- Using NullLogger ---\n");

        Application app2 = new Application(new NullLogger());
        app2.run();
    }
}
```

Output:

```
LOG: App started.
Running application...
LOG: App ended.

--- Using NullLogger ---

Running application...
```

Example2: Customer Lookup

```
package com.mannemlokesesh.example2;
```

```
interface Customer {
    boolean isNull();

    String getName();
}

class RealCustomer implements Customer {
    private String name;

    public RealCustomer(String name) {
        this.name = name;
    }

    public boolean isNull() {
        return false;
    }

    public String getName() {
        return name;
    }
}

class NullCustomer implements Customer {
    public boolean isNull() {
        return true;
    }

    public String getName() {
        return "Not Available";
    }
}

class CustomerFactory {
    private static final String[] names = { "Lokesh", "Kiran", "Anu" };

    public static Customer getCustomer(String name) {
        for (String n : names) {
            if (n.equalsIgnoreCase(name)) {
                return new RealCustomer(name);
            }
        }
        return new NullCustomer();
    }
}

public class CustomerClient {
    public static void main(String[] args) {
        Customer c1 = CustomerFactory.getCustomer("Lokesh");
        Customer c2 = CustomerFactory.getCustomer("Unknown");

        System.out.println("Customer 1: " + c1.getName());
        System.out.println("Customer 2: " + c2.getName());
    }
}
```

Output:

```
Customer 1: Lokesh
```

Customer 2: Not Available

Example3: Chain of Null-Aware Handlers

```
package com.mannemlokes.example3;

interface Handler {
    void handle(String request);
}

class ConcreteHandler implements Handler {
    public void handle(String request) {
        System.out.println("Handling: " + request);
    }
}

class NullHandler implements Handler {
    public void handle(String request) {
        // silently ignore
    }
}

public class Processor {
    private Handler handler;

    public Processor(Handler handler) {
        this.handler = handler;
    }

    public void process(String request) {
        handler.handle(request);
    }

    public static void main(String[] args) {
        Processor p1 = new Processor(new ConcreteHandler());
        p1.process("Data 1");

        Processor p2 = new Processor(new NullHandler());
        p2.process("Data 2");
    }
}
```

Output:

Handling: Data 1

27.9 Null Object vs Optional vs Null Check

Feature	Null Object	Optional	Manual Null Check
Type-safe	✓	✓	✗
Reusable	✓	✗	✗
Readable Code	✓	✓	✗

27.10 Pros and Cons



Pros

- Avoids null and NullPointerException
- Simplifies client code
- Implements polymorphism



Cons

- Can hide real bugs
- May add too many small classes

27.11 When to Use / Not to Use

Use When:

- You return null often
- You want to follow fail-safe defaults

Avoid When:

- You need to distinguish between null and default
- Performance/memory is critical

27.12 Summary

Feature	Description
Intent	Avoid null checks via empty objects
Use Case	Logging, Lookups, Chains
Key Benefit	Simpler, safer code

The **Null Object Pattern** is ideal when you want to avoid null and return an object with neutral behavior.